

Integration Guide

For an engineer at a common data environment — Asite, Procore, Dalux, or your own — who has to make this viewer open your customers' documents.

Read section 1 before planning anything. All four exchanges an integration needs are built, you can run a working host application from our repository today, and a stock install now serves the embed document and carries the frame-ancestors allow-list to put your origins on. **What has not been exercised end to end is a real document opening inside a real host frame**, so treat section 4 as the contract and your first integration as the thing that proves it. This guide says which parts you can build on now and which are newer than anyone's production traffic.

Companion document: **Technical Architecture**, for how the thing works internally.

Contents

1. What you can build against today

HOST CDE	EXCHANGE	VIEWER PRODUCT
Their web interface where the viewer appears	1. mount the viewer → ✓ BUILT iframe + cde.viewer.v1	/embed route no session, no auth guard
Their identity provider Entra / Okta / own	2. identify the user → ✓ BUILT host.init.identity — 3 untrusted fields	viewer-core 15 files · 6 services · 6 components
Their document store SharePoint / S3 / Blob	3. hand over the document → ✓ BUILT host.init.document, or POST /api/conversions	Conversion pipeline fetch → scan → render
Their markup store they stamp the author	← 4. return what was drawn ✓ BUILT 3 markup events + 1 operation pair	host-channel origin-checked in both directions
	⚠ NOT YET DEPLOYABLE the allow-list exists; nothing serves the embed document	

Figure 1 — All four exchanges now have code on both sides, checked against each other by a demo host written in plain JavaScript. The frame-ancestors allow-list the backend refused to have now exists — `cde.web.embed-parent-origins`, closed until a deployment names a host. What is still missing is a tier that serves the `/embed` document at all: the backend image carries no frontend, and the Kubernetes manifests route everything to the backend.

Two things are true at once, and conflating them will cost you a sprint.

The protocol is real. `cde.viewer.v1` is implemented on both sides, and our repository ships a host application — plain HTML and JavaScript, no framework, no build step — that frames the viewer, drives the handshake, stores markup, and refuses an operation. You can clone it, run it, and read it as the thing you are about to write. Section 5 tells you how.

The deployment now exists, and is new. The image serves the viewer’s pages itself when `cde.web.app.path` points at a staged build, so one container answers both `/embed` and the API, and emits the Content-Security-Policy that governs the document it just served. `cde.web.embed-parent-origins` names the origins permitted to frame that route and is closed until someone sets it — so ask your deployment contact to add your origin before you test, because until they do the frame comes up blank with an error only your browser console will show you.

What that deployment has and has not been through. Every page route has been served from a production build under the real policy and loaded in a browser with no content blocked. What has not happened is a document opening inside a host’s frame against an installed deployment — the pieces are each verified and the whole is not. Expect to be the first to find whatever that turns up, and plan a spike rather than a delivery date.

The conversion API has no such caveat. Section 3 is a complete, working integration you can ship against today, inside your own interface, with your own rendering — and for a “preview any file format” feature that is often the whole requirement.

2. The shape of it

Your CDE keeps its documents and its users. The viewer borrows a document briefly, converts it, and discards it.

We never hold your credentials and never learn which storage platform your link points at. SharePoint, S3, Azure Blob and GCS all reduce to “a URL that works for fifteen minutes”, which is the property that makes this portable across CDEs rather than a per-vendor connector.

3. Converting a document from your storage

#	STAGE	WHAT HAPPENS
1	Host mints a link	Short-lived, single-object, your credentials. We never hold them.
2	Submit	POST /api/conversions { sourceUrl, targetFormat } — 202 + Location header, under one second
3	Destination policy	Resolves the hostname and validates the ADDRESS, not the string. Refuses loopback, link-local, 169.254.169.254, RFC 1918, .internal. Redirects are NOT followed.
4	Bounded fetch	Size cap enforced during the stream, not after. Connect and read timeouts.
5	Quarantine + scan	Magic-byte type check, then ClamAV. Not referenced by the application until clean.
6	Convert	Sandboxed, resource-limited, network-isolated worker, out of process.
7	Poll	GET /api/conversions/{jobId} — PENDING, RUNNING, then SUCCEEDED / FAILED / CANCELLED (terminal)
8	Collect	GET /api/conversions/{jobId}/content — streamed

Figure 2 — The conversion API, end to end. Stages 1, 7 and 8 are yours; 2 to 6 are ours. Always send an Idempotency-Key so a timed-out retry returns the same job rather than converting twice. An embedded viewer opening a PDF skips this path entirely — the browser fetches your URL directly and no server of ours sees the file.

3.1 Mint the link

Generate a short-lived, single-object download URL with your own credentials — a Microsoft Graph download URL, an S3 presigned GET, an Azure blob SAS, a GCS signed URL. Keep the expiry tight; we fetch once, immediately.

Do **not** send us a permanent URL, a URL that needs a credential we would have to store, or a path on your internal network — section 3.5 explains what happens.

3.2 Submit

```
curl -X POST https://viewer.example.com/api/conversions \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: 8f14e45f-ceed-467a-9f3a-1d2c9b7e4a51" \
  -d '{
    "sourceUrl": "https://files.example.test/drawings/site-plan.dwg?token=synthetic",
    "targetFormat": "PDF"
  }'
```

```
202 Accepted
{ "jobId": "3f2a71c4-9b0e-4a2d-8c11-5e7d9a1b3f40", "status": "PENDING",
  "sourceHost": "files.example.test", "targetFormat": "PDF" }
```

Both fields are required. `targetFormat` accepts PDF and nothing else today — it is in the contract so that adding a second output is additive rather than breaking. The `jobId` is a UUID; use it verbatim on every other endpoint.

Requires the `document:convert` permission. Returns in under a second — the work has not happened yet, and that is the design: an endpoint either answers in under a second or hands back a job id in under a second.

Always send Idempotency-Key. A timed-out retry then returns the same job rather than converting the same file twice, and conversion is the expensive operation in this system.

Note what is not echoed back: the URL. Only its host is kept on the job record. The link is a bearer credential, so it is never written to the database, a log, or a message — which also means you cannot read it back to find out what you sent.

3.3 Poll

```
curl https://viewer.example.com/api/conversions/3f2a71c4-9b0e-4a2d-8c11-5e7d9a1b3f40 \
-H "Authorization: Bearer $TOKEN"
```

```
200 OK
{ "jobId": "3f2a71c4-9b0e-4a2d-8c11-5e7d9a1b3f40", "status": "SUCCEEDED",
  "sourceHost": "files.example.test", "sourceFileName": "site-plan.dwg",
  "targetFormat": "PDF", "resultSizeBytes": 486213, "failureReason": null }
```

PENDING while queued, RUNNING while working, then one of SUCCEEDED, FAILED or CANCELLED.

Those three are terminal and never change again, so stop polling when you see one. On FAILED, read `failureReason`.

There is no progress percentage and no sub-stage — the job reports the state it is in, not how far through it is. Back off as you poll; a large IFC will not finish in the first second, and hammering the endpoint only costs you rate limit.

3.4 Collect

```
curl https://viewer.example.com/api/conversions/3f2a71c4-9b0e-4a2d-8c11-5e7d9a1b3f40/content \
-H "Authorization: Bearer $TOKEN" -o converted.pdf
```

Streamed, with `Content-Disposition: attachment` and `X-Content-Type-Options: nosniff`. Cancel an in-flight job with `DELETE /api/conversions/{jobId}`.

3.5 What we refuse, and why

Your URL is dereferenced by our server, which makes it an SSRF vector. Every submission goes through a destination policy that:

- **resolves the hostname and validates the resolved address**, not the string — a name that resolves to 169.254.169.254 is refused however it is spelled;
- refuses loopback, link-local, RFC 1918 (10/8, 172.16/12, 192.168/16), `::1`, and `.internal` names;
- **does not follow redirects at all**. Not “follows a capped number and re-validates” — none. Your link must resolve to the bytes directly;
- requires https unless the deployment has explicitly opted out;
- enforces the deployment’s list of permitted storage hosts, where one is configured;

- bounds the fetch: size enforced **during** the stream, plus connect and read timeouts.

The redirect rule is the one most likely to break you. Plenty of storage 302s to a CDN. Test the exact URL you intend to send, not a URL that eventually reaches the bytes. And ask which hosts are allow-listed on the deployment before you integrate.

After the fetch: content type is decided by **magic bytes**, never by your Content-Type header or the file extension; the file lands in a quarantine prefix; ClamAV scans it; only then does it convert. An infected file is deleted and the event audited.

3.6 Errors

RFC 9457 problem documents throughout, with a `traceId` you can quote.

```
{ "type": "/problems/fetch-not-permitted",
  "title": "Fetch not permitted",
  "status": 422,
  "detail": "The source URL resolves to a private address (10.0.4.19).",
  "traceId": "4f8a1c2e9b7d6a5f3e2d1c0b9a8f7e6d" }
```

Endpoint	Status	Means
POST /api/conversions	202	Accepted. The Location header names the job to poll — use it rather than assembling the URL yourself
	422	The link may not be fetched: wrong scheme, an unlisted host, or a refused destination. This is the SSRF refusal — it is a 422, not a 403
	429	The conversion queue is full, not a per-caller rate limit. Retry after the interval in Retry-After
GET /api/conversions/{jobId}	404	No such job is visible to you. Note the wording: another tenant's job is invisible, not forbidden
DELETE /api/conversions/{jobId}	404	As above
GET /{jobId}/content	409	The job has not succeeded, so there is nothing to download yet. Poll to a terminal state first

Table 1 — Status codes you will actually meet.

Read `detail` and show it. It is written to be shown. Quote `traceId` to support — it is the only thing that ties your failure to our logs.

4. Formats

Input	Status	Route	Notes
PDF	Supported	Passed through; pdfjs-dist	OCR available for scanned pages, adds an invisible text layer
DXF	Supported	ezdxf → SVG or PDF	Text is real and searchable in both
Office	Supported	LibreOffice → PDF	docx, xlsx, pptx
IFC	Supported	Geometry + hierarchy tree	Tree is the primary interface, not a fallback
DWG	Needs ODA	ODA → DXF, then as above	The image ships no DWG reader.

Input	Status	Route	Notes
			The operator supplies an ODA File Converter — see below
DWF	Not supported	—	No route exists. Autodesk's Design Web Format is not read by any component of the pipeline
RVT / RFA	Not supported	Detected and refused	Recognised by OLE2 magic bytes and rejected with REVIT_BINARY. Identified only so the error is clean

Table 2 — What the pipeline actually accepts. Four supported outright, one conditional, two not supported at all.

DWG now requires an ODA File Converter that the operator supplies. This changed, and in your favour if you were reading the previous edition. That edition said DWG worked in our hosted service and was unshippable on-premises, because the image bundled LibreDWG (GPL-3.0) and distributing it owed recipients corresponding source we did not provide. **LibreDWG has been removed.** Nothing encumbered is distributed, and the same rule now applies everywhere: mount an ODA installation, point ODA_PATH at it, and DWG works — hosted or on-premises, no distinction. Without one, the deployment has no DWG reader at all. ODA is licensed by you or your operator directly from the Open Design Alliance; check `odaRunnable` rather than `odaInstalled` in the status response, because a mount missing its libraries or its execute bit reports as present and is useless.

Only PDF opens inside an embedded frame today. Everything in this table is supported through the conversion API in section 3. The embed reaches the browser-rendered path only, so an embedded viewer asked for a `.docx` or an `.ifc` returns a 415 naming the format and saying the conversion service is what is missing. If your embedded use case is not PDF-first, raise it — the work is wiring, not design.

4.1 Why RVT and DWF are absent, and what it would take

These two are named separately because they are the ones most often assumed, and because the reason they are missing is not “nobody got to it yet”.

RVT (and RFA) is a proprietary Autodesk binary with no open reader. The pipeline detects it by OLE2 magic bytes and refuses it deliberately, so you get a clean `REVIT_BINARY` error rather than a corrupt render. Every route to supporting it is commercial.

Route	What it costs
Autodesk Platform Services (formerly Forge)	A cloud API. It SENDS THE MODEL TO AUTODESK, which collides directly with the data-residency and sovereignty position — for a Defence or IRAP-scoped tenant it is not merely a cost question, it is prohibited
Revit itself, plus an export plugin	Windows hosts, a licence per seat, and a rendering farm that is not the product's architecture
A commercial SDK (ODA BimRv or equivalent)	A vendor relationship and a per-deployment fee. ADR 13 settled the same question for DWG by making ODA operator-supplied rather than bundled, so the shape of the answer is known — it is a commercial conversation, not an open one

Table 3 — The three routes to RVT, and why none is free.

The sovereignty collision is the important one. **The cheapest route is the one we can least use**, because the tenants most likely to hold Revit models are also the ones whose data must not leave the jurisdiction.

DWF is unimplemented rather than refused. It is a ZIP container holding W2D and W3D streams, and reading it is ordinary engineering rather than a licence problem — but no component of the pipeline knows the format today, so a DWF submission will fail as an unrecognised type. If DWF matters to you, say so: it is schedulable work in a way RVT is not.

If you need either, raise it as a commercial requirement, not a bug. The answer involves a vendor contract and, for RVT, a residency decision that engineering cannot take alone.

5. Embedding — built, and what your deployment must configure

An iframe and a versioned postMessage protocol. The viewer authenticates nobody and authorises nothing: you mint a short-lived URL for the document, tell it a name to display, and receive what the user did — every operation that changes anything comes back to you to authorise server-side.

Recorded as ADR 14 (cde-platform, docs/adr/0014-viewer-embed-and-identity-contracts.md), with the message-level detail in docs/viewer-embed-protocol.md. Both sides are now implemented, and the protocol document is the normative one — where it and this guide disagree, it wins.

5.1 Run the demo host first

demo/ in the frontend repository is a host application that frames the viewer. It is not the viewer, and that is the point: **it is the side you are about to write**, in plain HTML, CSS and JavaScript with no framework, no build step and no dependencies, because your stack is your business.

```
npm start          # the viewer, on :4200
node demo/server.mjs  # the host, on :4401

# then open http://localhost:4401
# viewer somewhere else?
VIEWER_ORIGIN=https://viewer.example node demo/server.mjs
```

Two ports deliberately: a different port is a different origin, so the demo exercises the real cross-origin path — origin checks on every message, CORS on the document fetch, frame-ancestors on the viewer. A demo served from one origin would pass with every one of those broken.

It ships three generated sample documents and a message log down the side, so the handshake is visible as it happens. Worth trying in this order: open the drawing and watch viewer.ready → host.init → viewer.loaded; draw on it and see the markup appear in the host's store; reload the page and reopen the document, and watch the markup come back — the viewer forgot, the host remembered; change the display name and draw again, and see the new name stamped on a markup the viewer never put a name on. Then tick document:sign and use Sign: the control renders and the operation is **refused**, and both of those are correct.

5.2 The one thing your viewer deployment must change

A deployment that has not been told about you refuses your iframe, and the failure looks like a blank frame rather than an error. `frame-ancestors` is 'none' until your origins are named, deliberately: opening the embed is a configuration act and never a default. This is not something you can set from your side.

The setting is `cde.web.embed-parent-origins` — a list of exact origins, 'none' when empty, and validated at startup. Wildcards, the null origin, CSP keywords and anything carrying a path are all refused by name, because **a frame-ancestors source a browser cannot parse is not a closed door**: the browser drops what it cannot read and applies the rest, so a typo widens the policy rather than breaking visibly. Give your deployment contact the exact origins you will frame from, including scheme and any non-default port.

This header is the **authorisation** decision about who may frame the viewer. The `parentOrigin` you configure in the handshake is only addressing — it says where the viewer should post, not who is permitted to frame it. A deployment that relaxed only the second would be framable by anyone who sent the right message.

Ask your deployment contact which component serves `/embed`. The setting governs what the *backend* answers. A stock install now serves the embed document from the backend image itself, so one component emits the document and the header together and there is nothing else to configure. But a deployment that puts the Angular build behind its own web tier has that tier serving the document, and **it needs the same `frame-ancestors` value** — the backend's copy governs only what the backend answers. Two questions, not one: which of your origins are on the allow-list, and what serves the document.

5.3 The minimum integration

Four steps, and none of them involve a token.

- **Frame the embed route** and listen for messages, checking `event.origin` against the viewer origin on every single one — not once at setup.
- **Wait for `viewer.ready`**, then post `host.init` with the document (a short-lived URL, its media type, a display name, and your own `externalId`) and, if the user may do more than read, an identity block.
- **Store what comes back.** `viewer.markupCreated`, `...Updated` and `...Deleted` carry the markup; `shapeData` is an opaque string you store and hand back, never parse. Stamp the author yourself from the session you already hold.
- **Listen to as much of the rest as you need, and none of it if you do not.** Section 5.4 lists the lifecycle events. Every one is optional.
- **Answer `viewer.operationRequest`** with `host.operationResult` — applied, refused or failed. One message pair covers all seventeen document operations; they are not seventeen message types.

Send no credential in any message, in either direction. The document URL is a bearer credential with a short life and is the only thing resembling one that crosses the boundary; it is never stored, logged, or echoed back to you. Never post to `'*'`.

One subtlety worth internalising early: **a refused must be possible even for an operation whose capability you granted in `host.init`**. Capabilities decide which controls render; authorisation happens when the operation is requested. If the two ever disagree — the user's permission changed thirty seconds ago — the refused is correct and the rendered button was merely stale.

Three operations never become viewer-side, whatever else changes: `document.sign`, `version.create` and `version.restore`. A signature over a copy you have since replaced is worse than no signature, and the viewer cannot know whether its copy is still current.

One constraint from the previous edition of this guide is unchanged and still catches people: **the CORS source registers no origin at all unless the deployment names one in full**. For a PDF, the browser fetches your URL directly, so your storage must allow the viewer's origin to read it.

5.4 Events you can hook, and what each is honestly for

Fourteen message types run viewer → host. Four exist purely so you can **record** what a reader did without polling us or inferring it from markup traffic — and the inference is wrong in ways that are not obvious, which is why they exist.

You want to know	Listen to	What to watch out for
A document was opened	<code>viewer.opened</code>	Fires before the viewer knows whether it can render the file, and before any fetch. Exactly one of <code>viewer.loaded</code> or <code>viewer.error</code> follows it
It rendered	<code>viewer.loaded</code>	Carries <code>pageCount</code> , <code>mediaType</code> and what rendered it
Your stored markup arrived	<code>viewer.markupLoaded</code>	Check rejected. Non-zero means your store holds markup the viewer cannot draw — a data problem on your side that was previously silent
The user drew, changed or removed something	<code>viewer.markupCreated</code> / <code>...Updated</code> / <code>...Deleted</code>	No <code>author</code> field. You stamp it from the session you already hold (5.5)
The user selected a markup	<code>viewer.selectionChanged</code>	<code>markupId</code> , or null when nothing is selected
Which pages were actually read	<code>viewer.pageRendered</code>	Once per page per document. Zooming repaints a page and does NOT re-announce it, which is what makes a "pages read" count mean anything
Where the user is now	<code>viewer.viewChanged</code>	Throttled to 4/s. Use this to follow a reader live; use <code>pageRendered</code> to record what was seen
A document stopped being shown	<code>viewer.unloaded</code>	Carries the id of the document that CLOSED, not the one arriving. Not guaranteed on teardown — see below

Table 4 — The events worth hooking, and the trap in each.

Do not treat `viewer.unloaded` as a guarantee. If your page removes the `iframe`, navigates away, or the tab closes, nothing can post from a frame that no longer exists. Use it to close a record you are already keeping, never as the only place you write one — the same caveat that applies to `beforeunload` in your own page, for the same reason.

Two more sharp edges worth knowing before you build against these. A document the viewer **rejects outright** — a descriptor missing `mediaType`, say — produces `viewer.error` with no `viewer.opened` before it, because there was no document to open. And a

`host.loadMarkup` whose markup is not an array is **ignored silently and acknowledges nothing**: if you get no `viewer.markupLoaded`, check you sent an array.

All four were added after the protocol shipped, which is the compatibility promise working as intended — new message types are additive within v1, so an integration written before them keeps running and simply never registers a handler. Full payloads: §6.3 of `docs/viewer-embed-protocol.md`.

5.5 Identity: there is nothing to configure

Not “you need to set up SSO”. The embed path has no identity system to configure at all. Our own `/api/annotations` still keys a markup’s author to a row in our user table — integrating through *that* API would mean shadow-provisioning your users into our database, a data-protection conversation before it is an engineering one. The embed never reaches it.

THE PLATFORM’S OWN API — unchanged	THE EMBED PATH — built
Priya, at Asite host identity	Priya, at Asite known only to Asite
↓ blocked	↓
Annotation.author @ManyToOne User	host.init.identity display name · subjectId · capabilities
↓	↓
OUR users table she must exist as a row	Nothing persisted the viewer stamps no author
Integrating through this API means shadow-provisioning every host user into our database.	The host stays the system of record, and stamps the author itself when it stores the markup.

Figure 3 — Both paths exist in the code. `/api/annotations` still keys an author to a row in our own user table, and always will for this platform’s own users. The embed path never reaches it: the viewer emits a markup with no author at all, and the host attributes it from the session it already holds.

Three untrusted fields, and that is the whole of it. You send a display name, an opaque subject id, and a list of capabilities. All three are used for presentation and for deciding which controls render; none is trusted for authorisation, because the browser is not a place authorisation happens. The author is stamped by whoever persists the markup — you. Nothing about your users reaches our database.

You do **not** need to mint a JWT, and you do not need an identity provider we can talk to. If an earlier version of this guide told you to prepare a signed assertion with a stable subject claim, disregard it — that was written while the question was open, and the answer went the other way. **Sending no identity at all is a supported read-only deployment**, not a degraded one.

One consequence to plan around: **live collaboration is not available in an embed**. Cursors and presence ride a socket authenticated by a viewer session, and an embedded viewer has no session by design. That is the acknowledged cost of “the viewer authenticates nobody”, and it is unsolved rather than decided against.

6. What is settled, and what is still open

Everything in the first list is decided and implemented — design against it. Everything in the second is a real gap we would rather you heard from us than discovered.

Settled

- **Embedding is an iframe** with a per-tenant frame-ancestors allow-list. Not a web component: the viewer renders untrusted documents, and sharing your origin would mean a document that escapes the PDF renderer runs with your session on your domain.
- **Identity is three untrusted fields** — a display name, an opaque subject id, and capability flags that decide which controls render and nothing else. No signed assertion, no viewer session, no JWT.
- **Markup comes back as events**, with the geometry as an opaque string you store and hand back. There is no author field — you stamp that.
- **Document identity is yours**. Pass an externalId in the handshake and it comes back on every event, so you never hold a map between your id and ours.
- **One message pair covers all seventeen document operations**, and three of them — sign, create version, restore version — will never move to the viewer.
- **The protocol version is in every message, and viewer.ready tells you which versions the deployment speaks**. Read it and pick; do not assume. Within v1 we may add message types, optional fields, commands and operations — you ignore what you do not recognise, which is why the rule is *validate*, not *reject on unknown field*. Removing anything needs v2, announced with at least six months' notice and both versions running over the overlap.

Still open

- **Nobody has opened a document in a host frame on an installed deployment** — section 5.2. Every piece is verified and the whole has not run. Plan a spike.
- **Non-PDF formats in an embedded frame** — section 4. Wiring, not design.
- **Collaboration in an embed** — section 5.5. Genuinely unsolved.
- **Accessibility evidence**. See section 7 before you rely on ours.

Full message shapes: docs/viewer-embed-protocol.md. Where this guide and that document disagree, that document is the normative one.

7. Obligations you inherit

Accessibility. WCAG 2.2 AA is a procurement gate in the UK, EU, Australia and the US, and an embedded viewer becomes part of *your* conformance claim. If you embed our WebGL model view, you inherit the requirement for an equivalent accessible route to the same information — we provide the hierarchy tree for exactly this reason, and it needs to remain reachable in your integration. Exports must be tagged and PDF/UA-conformant.

Do not inherit a conformance claim from us, because we are not making one yet. An accessibility statement, a VPAT 2.5 INT conformance report and a screen-reader matrix all exist in our repository, and all three record the same thing: no criterion has been evaluated by test. Real work has been done — keyboard-reachable controls, a visible focus indicator throughout, prefers-reduced-motion honoured, an authentication flow that meets SC 3.3.8 by construction — but none of it has been through an audit, and

an untested claim is worth nothing in a procurement. If your bid depends on ours, ask for the current state in writing rather than citing this guide.

Trademarks. “Works with Microsoft SharePoint” is nominative fair use. “Microsoft-approved”, their logo, or any implication of partnership is not. The same applies to Amazon, Google and Autodesk. Integration documentation is where this goes wrong most easily, in both directions.

Certification claims. Do not describe this product as SOC 2, ISO 27001 or IRAP certified. Those certifications are not currently held, and stating otherwise is a misrepresentation with regulatory consequences rather than a marketing stretch.

8. Integration checklist

Today

- Can you mint a short-lived, single-object download URL from your storage?
- Does that URL resolve to the bytes directly, with no redirect?
- Do you have somewhere to hold a job id between submit and collect?
- Do you generate an Idempotency-Key per logical submission?
- Do you back off while polling, and honour Retry-After on 429?
- Do you surface detail and traceId from problem documents to your support path?
- Have you confirmed your DWG position?

Before you scope an embedded integration

- Have you run the demo host and watched a handshake? (demo/README.md)
- Have you read the embed protocol? (docs/viewer-embed-protocol.md)
- **Have you given your deployment contact the exact origins you will frame from, and confirmed they are on `cde.web.embed-parent-origins`?**
- **Have you confirmed what will serve the `/embed` document on that deployment?**
- Can you mint a short-lived URL for a document, and serve a page that frames us?
- Does your storage allow the viewer’s origin to read that URL from the browser?
- Do you check `event.origin` on every message, not once at setup?
- Do you have somewhere to store markup that is not the rendered file, and do you stamp the author from your own session rather than from the message?
- Do you know which of the 17 document operations you need to implement as callbacks, and can you return refused as readily as applied?
- Is your embedded use case PDF-first, or do you need the conversion path wired in?

The integration you can ship against a stock deployment today is still section 3. The embed is built and demonstrable, and it needs one header changed on the viewer side before it runs anywhere but a development install. Scope it as something to plan with us rather than something to build against unilaterally — and if all you need is viewing, converting through section 3 and rendering the result yourself remains a complete feature with none of these caveats.